DOC Genie

Project Description

Doc-Genie is a streamlined Python documentation tool that leverages Abstract Syntax Tree (AST) analysis to provide automatic docstring generation through an interactive, user-friendly Gradio web interface. The platform addresses the tedious nature of maintaining Python documentation by delivering automated function analysis and consistent Google-style docstring generation, ensuring your codebase remains well-documented and highly readable.

Using robust native AST parsing algorithms, the system effectively analyzes raw Python code to identify function definitions, safely extracting function signatures, names, and parameters. It then automatically constructs formatted Google-style docstrings and precisely injects them back into the original code with correct indentation. This ensures syntactically safe modifications without disrupting the existing logic or structure of the codebase.

Doc-Genie transforms Python documentation from a manual chore into a quick, hassle-free experience. Through its modern Gradio interface, developers can simply paste their raw Python code and instantly receive a fully documented version, encouraging best practices, maintaining consistency across scripts, and saving valuable development time

Scenarios

Scenario 1: Legacy Script Modernization A developer inherits an older Python script containing numerous functions without any documentation. Instead of manually typing out docstring formats for each one, they paste the code into Doc-Genie. The AST-based tool instantly identifies all function definitions and argument names, safely injecting perfectly formatted Google-style docstring templates. This provides a clean, organized starting point for the developer to fill in the functional details, saving time on structural boilerplate.

Scenario 2: Rapid Prototyping Documentation A software engineer is rapidly building a prototype and writing utility functions without pausing to document them. Before sharing the code with a colleague, they use Doc-Genie's Gradio web interface. The tool quickly parses the raw Python code and inserts comprehensive docstring skeletons for every function. The engineer then rapidly fills in the "Description" and "Type" placeholders, ensuring the shared code looks professional and is easy for their colleague to understand.

Scenario 3: Educational & Training Purpose An instructor teaching Python programming uses Doc-Genie to demonstrate industry-standard documentation practices. Students paste their newly written functions into the tool, which automatically generates Google-style docstrings based on their code's structure. This hands-on visualization helps students understand what a structured professional docstring (with standard "Args" and "Returns" sections) should look like, reinforcing good coding habits from the start.

Scenario 4: Ensuring Consistent Team Formatting A small development team wants to ensure all members use the exact same docstring format. Before committing code to their repository, team members use Doc-Genie to generate their initial docstrings. Because Doc-Genie relies on structural AST parsing rather than manual typing, every function across the project gets an identically formatted and perfectly indented Google-style template. This eliminates formatting inconsistencies during code reviews.

Scenario 5: Preparing Open Source Contributions A developer is preparing to contribute to an open-source Python project that strictly requires Google-style documentation. They paste their custom functions into Doc-Genie to automatically generate the required formatting structure based on their function signatures.

Architecture Overview

Doc-Genie - is built as a modular platform combining Python's AST module with Gradio for an intelligent documentation generation system. The architecture prioritizes code analysis accuracy, documentation quality, and user experience through advanced parsing techniques and comprehensive output options.

Core Technologies

- Python 3.8+: Core language with AST module for code analysis
- Gradio 4.0+: Interactive web interface framework
- AST (Abstract Syntax Tree): Python's built-in code parsing library
- ReportLab: PDF generation for comprehensive documentation reports
- Regular Expressions: Pattern matching for code analysis
- Google Colab: Cloud-based development and deployment environment
- HTML5/CSS3: Modern web standards for interface styling

Pre-requisites

Software Requirements

- **Python 3.8 or higher**: Required to run the application and access modern AST features. Downloadable from python.org.
- **pip (Python Package Installer)**: Required to install necessary third-party dependencies. (Included by default with standard Python installations).
- **Gradio Library**: The core third-party UI framework (pip install gradio).
- **Modern Web Browser**: Required to view and interact with the Gradio web interface (Google Chrome, Mozilla Firefox, Apple Safari, or Microsoft Edge).
- **Code Editor / IDE (Optional)**: Such as VS Code, PyCharm, or Jupiter Notebook/Collab for running and modifying the script locally.

Knowledge Prerequisites

- **Basic Python Programming**: Understanding of Python syntaxes, functions, variables, and how to execute a Python script (.py file).
- **Familiarity with Docstrings**: Basic understanding of what Python docstrings are (specifically Google-style documentation) and why they are used to document functions.
- **Command Line / Terminal Navigation**: Basic ability to open a terminal or command prompt to run scripts (e.g., executing python doc_genie.py).
- **Basic Package Management**: Knowing how to install Python libraries using pip.
- **Understanding of AST (Optional)**: While not required to *use* the application, a high-level understanding of Abstract Syntax Trees is helpful for developers looking to modify or extend the tool's underlying parsing logic.

Project Workflow

Step 1: Code Input & Interface Interaction

- The user launches the application, which opens the **Gradio interactive web interface** locally in their browser.
- The user pastes their undocumented or partially documented Python code into the "Paste Python Code" input textbox provided by the Gradio UI.
- The user clicks the "Submit" button to initiate the automated process.

Step 2: Signal Transmission

- The Gradio frontend captures the raw text input.
- The input is immediately passed to the backend
- ui_generate wrapper function, which forwards the raw code string to the core process_code engine.

Step 3: AST Parsing & Code Analysis

- The system utilizes Python's native ast.parse to convert the raw string code into an **Abstract Syntax Tree (AST)**.
- The system traverses this tree (ast.walk) to systematically identify and isolate specific code structures, specifically looking for function definitions (ast.FunctionDef).

Step 4: Automated Docstring Generation

- For every function definition found, the system extracts critical metadata: the **function name** and its **arguments/parameters**.
- This metadata is passed to the generate_docstring function.
- A structured, Google-style docstring template is dynamically generated, populating placeholder descriptions, arguments, and return types based on the extracted metadata.

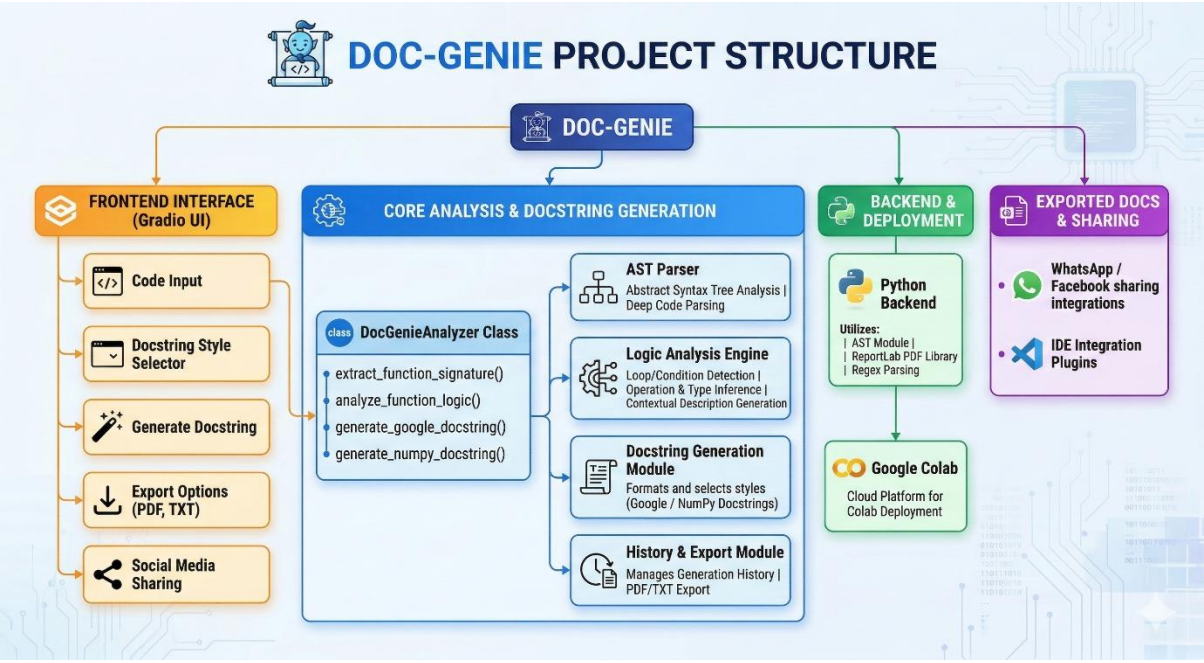
Step 5: Code Injection & Formatting

- The system determines the exact line number (lineno) and indentation level (col_offset) of the original function's body.
- The generated docstring is safely injected directly beneath the function definition, ensuring perfect alignment and indentation without disrupting the original syntax.

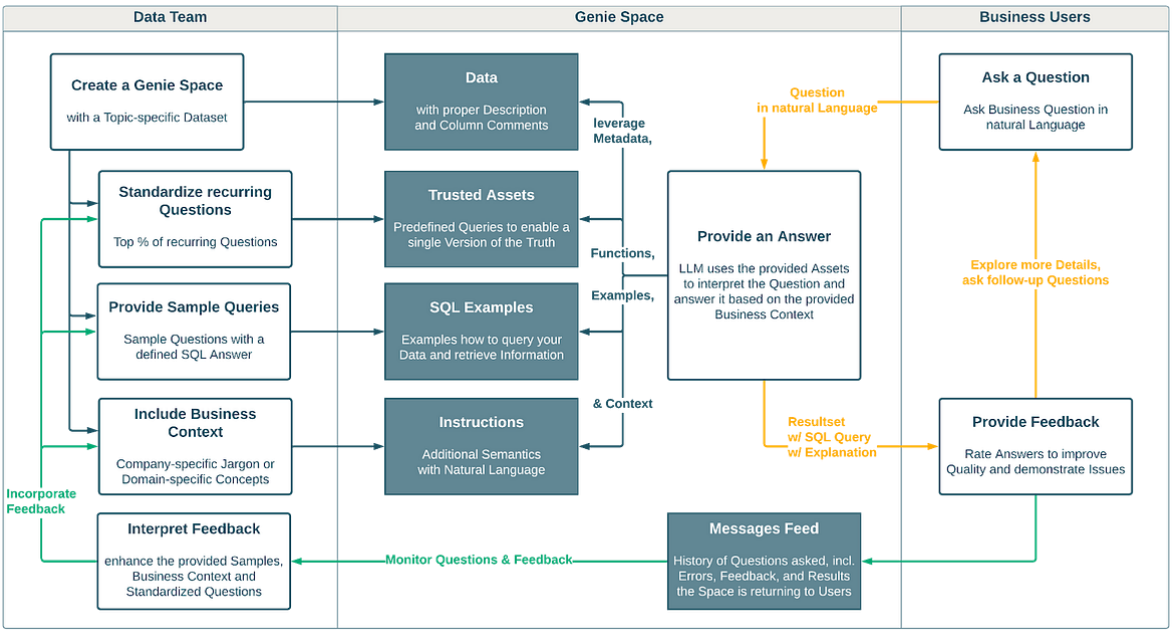
Step 6: Output & Delivery

- The system reconstructs the modified code back into a standard Python string.
- This fully documented code is sent back through the Gradio pipeline.
- The Gradio interface renders the final result in the "Code with Docstrings" output textbox, allowing the user to instantly review, copy, and use their newly documented code

Project Structure



Technical Architecture



Environment Setup & Core Architecture

Step 1 Install Dependencies

Create requirements.txt with the following dependencies:

- gradio==4.41.0
- transformers==4.35.0
- torch==2.0.0
- reportlab==4.0.7
- requests==2.31.0
- black==24.1.1
- autopep8==2.1.0

Install via: pip install -r requirements.txt

```
[7]
3s
!pip install -q gradio transformers torch black autopep8 reportlab requests ast-super-parser

import gradio as gr
from datetime import datetime
from pathlib import Path
import json
import re
import ast
from reportlab.lib.pagesizes import letter, A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Table, TableStyle, Preformatted
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
import requests
import urllib.parse
import inspect
```

Step 2 Initialize Core Analyzer Class

Create DocGenieAnalyzer class with following structure:

```
class DocGenieAnalyzer:
    @staticmethod
    def extract_function_signature(code):
    - Parse AST from code
    - Extract function name, parameters, return type
    - Return signature dictionary
```

@staticmethod
def analyze_function_logic(func_def, code):
- Walk through AST nodes
- Detect loops, conditions, operations
- Generate contextual description
- Return analysis dictionary

@staticmethod
def generate_google_docstring(signature, analysis, code):
- Format Google-style docstring
- Include summary, args, returns
- Return formatted string

@staticmethod
def generate_numpy_docstring(signature, analysis, code):
- Format NumPy-style docstring
- Include summary, parameters, returns
- Return formatted string

```python
class DocGenieAnalyzer:
    """Analyze Python functions and generate professional docstrings"""

    @staticmethod
    def extract_function_signature(code):
        """Extract function name, parameters, return type from code"""
        try:
            tree = ast.parse(code)
            func_def = tree.body[0]
            if not isinstance(func_def, ast.FunctionDef):
                return None, None

            func_name = func_def.name
            params = []

            for arg in func_def.args.args:
                param_type = "Any"
                if arg.annotation:
                    param_type = ast.unparse(arg.annotation) if hasattr(ast, 'unparse') else "Any"
                params.append({
                    'name': arg.arg,
                    'type': param_type,
                    'default': None
                })
```

```python
            return None, None

    @staticmethod
    def analyze_function_logic(func_def, code):
        """Deep analysis of function body to understand actual behavior"""
        try:
            body = func_def.body
            analysis = {
                'has_loops': False,
                'has_conditions': False,
                'has_return': False,
                'loop_types': [],
                'condition_types': [],
                'operations': [],
                'return_types': [],
                'variables_used': set(),
                'function_calls': [],
                'description': ""
            }

            # Walk through AST
            for node in ast.walk(func_def):
                if isinstance(node, ast.For):
                    analysis['has_loops'] = True
                    analysis['loop_types'].append('for')
```

```python
        elif isinstance(node, ast.While):
            analysis['has_loops'] = True
            analysis['loop_types'].append('while')
        elif isinstance(node, ast.If):
            analysis['has_conditions'] = True
            analysis['condition_types'].append('if')
        elif isinstance(node, ast.Return):
            analysis['has_return'] = True
            if node.value:
                analysis['return_types'].append(ast.unparse(node.value) if hasattr(ast, 'unparse') else 'value')
        elif isinstance(node, ast.BinOp):
            op = node.op
            if isinstance(op, ast.Mult):
                analysis['operations'].append('multiplication')
            elif isinstance(op, ast.Add):
                analysis['operations'].append('addition')
            elif isinstance(op, ast.Sub):
                analysis['operations'].append('subtraction')
            elif isinstance(op, ast.Div):
                analysis['operations'].append('division')
        elif isinstance(node, ast.Call):
            if isinstance(node.func, ast.Name):
                analysis['function_calls'].append(node.func.id)
        elif isinstance(node, ast.Name):
            analysis['variables_used'].add(node.id)

    # Build detailed description
    description_parts = []

    if func_def.name:
        description_parts.append(f"Executes {func_def.name} operation")

    if analysis['has_conditions']:
        description_parts.append("with conditional logic")

    if analysis['has_loops']:
        description_parts.append("and iterates through values")

    if 'multiplication' in analysis['operations'] and 'subtraction' in analysis['operations']:
        description_parts.append("performing mathematical calculations")
    elif 'multiplication' in analysis['operations']:
        description_parts.append("using multiplication operations")
    elif 'addition' in analysis['operations']:
        description_parts.append("using addition operations")
    elif 'division' in analysis['operations']:
        description_parts.append("using division operations")

    if analysis['return_types']:
        description_parts.append(f"and returns the computed result")

    analysis['description'] = " ".join(description_parts) + "."
    if len(analysis['description']) < 20:
        analysis['description'] = "Performs computation based on input parameters."

    return analysis
except Exception as e:
    return {
        'has_loops': False,
        'has_conditions': False,
        'has_return': False,
        'description': "Executes function logic based on provided parameters."
    }
```

Advanced AST Analysis Engine

Function Signature Extraction

Extract comprehensive function information:

Input: def calculate(x: int, y: float = 3.5) -> float:

Extracted Information:

```
{
    'name': 'calculate',
    'params': [
    {'name': 'x', 'type': 'int', 'default': None},
    {'name': 'y', 'type': 'float', 'default': '3.5'}
    ],
    'return_type': 'float'
}
```

Code reference (hint: click on image and you can copy paste from Comment)

Deep Logic Analysis

Analyze function behavior through AST walking:

Detection Capabilities:

1. Loop Detection: Identifies for/while loops
2. Condition Detection: Finds if/elif/else statements
3. Operation Detection: Recognizes mathematical operations
4. Return Analysis: Tracks return statements and types
5. Function Calls: Identifies internal and external calls
6. Variables: Tracks variables used in function

Analysis Output:

```
{
    'has_loops': True,
    'has_conditions': True,
    'operations': ['multiplication', 'addition'],
    'function_calls': ['helper_func'],
    'description': 'Executes calculation with conditional logic...'
}
```

Code reference (hint: click on image and you can copy paste from Comment)

Docstring Generation & Formatting

Google-Style Docstring Generation

Generated Google-style docstring example:

> """Calculate factorial of a number.
>
> Executes calculation with conditional logic and returns
> the computed result.
>
> Args:
> n (int): Parameter for controlling n behavior.
>
> Returns:
> int: The result of the computation.
> """

Format includes:

- Single-line summary
- Blank line separation
- Args section with types and descriptions
- Returns section with type and description

```python
    @staticmethod
    def generate_google_docstring(signature, analysis, code):
        """Generate accurate Google-style docstring"""
        summary = analysis.get('description', 'Processes input and returns result.')

        docstring = f'    """{summary}\n'

        if signature['params']:
            docstring += '\n    Args:\n'
            for param in signature['params']:
                default_str = f" (default: {param['default']})" if param['default'] else ""
                docstring += f"        {param['name']} ({param['type']}): Parameter for controlling {param['name']} behavior{default_str}.\n"

        docstring += f"\n    Returns:\n        {signature['return_type']}: The result of the computation.\n"
        docstring += '    """\n'

        return docstring
```

NumPy-Style Docstring Generation

Generated NumPy-style docstring example:

```
"""Calculate factorial of a number.

Executes calculation with conditional logic and returns
the computed result.

Parameters
----------
n : int
Parameter for controlling n behavior.

Returns
-------
int
The result of the computation.
```

```python
    @staticmethod
    def generate_numpy_docstring(signature, analysis, code):
        """Generate accurate NumPy-style docstring"""
        summary = analysis.get('description', 'Processes input and returns result.')

        docstring = f'    """{summary}\n'

        if signature['params']:
            docstring += '\n    Parameters\n    ----------\n'
            for param in signature['params']:
                default_str = f", default: {param['default']}" if param['default'] else ""
                docstring += f"    {param['name']} : {param['type']}{default_str}\n"
                docstring += f"        Parameter for controlling {param['name']} behavior.\n"

        docstring += f"\n    Returns\n    -------\n    {signature['return_type']}\n"
        docstring += f"        The result of the computation.\n"
        docstring += '    """\n'

        return docstring

analyzer = DocGenieAnalyzer()
generation_history = []
```

```python
def generate_docstring(code, style="google"):
    """Generate docstring for Python function with accurate analysis"""

    if not code.strip():
        return None, "❌ Empty code provided!"

    try:
        signature, func_def = analyzer.extract_function_signature(code)
        if not signature:
            return None, "❌ Invalid Python function code!"

        # Deep analysis of the function
        analysis = analyzer.analyze_function_logic(func_def, code)

        if style.lower() == "google":
            docstring = analyzer.generate_google_docstring(signature, analysis, code)
        else:
            docstring = analyzer.generate_numpy_docstring(signature, analysis, code)

        # Extract function definition line
        code_lines = code.split('\n')
        func_line = code_lines[0]

        # Build final function with docstring
        final_code = func_line + '\n' + docstring
        for line in code_lines[1:]:
            final_code += line + '\n'

        generation_history.append({
            'code': code,
            'docstring': docstring,
            'style': style,
            'timestamp': datetime.now().isoformat(),
            'analysis': analysis
        })

        return final_code, f"✅ {style.upper()} Docstring Generated!"

    except Exception as e:
        return None, f"❌ Error: {str(e)}"

def process_upload(file):
    """Process uploaded Python file"""
    if not file:
        return None, "❌ No file uploaded!"

    try:
        content = file.read().decode('utf-8')
        return content, "✅ File loaded successfully!"
    except:
        return None, "❌ Failed to read file!"

def clear_history():
    """Clear generation history"""
    global generation_history
    generation_history = []
    return "✅ History cleared!"

def download_pdf():
    """Download all generated docstrings as PDF"""
    if not generation_history:
        return "❌ No documentation generated!", None

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.pdf"
```

```python
    try:
        doc = SimpleDocTemplate(filename, pagesize=A4, topMargin=0.5*inch, bottomMargin=0.5*inch)
        styles = getSampleStyleSheet()
        story = []

        title_style = ParagraphStyle(
            'Title',
            parent=styles['Heading1'],
            fontSize=24,
            textColor='#FF6B6B',
            spaceAfter=20,
            alignment=1
        )

        code_style = ParagraphStyle(
            'Code',
            parent=styles['Normal'],
            fontSize=9,
            fontName='Courier',
            backColor='#F0F0F0',
            borderColor='#CCCCCC',
            borderWidth=1,
            borderPadding=6
        )

        story.append(Paragraph("📚 Doc-Genie Documentation Report", title_style))
        story.append(Paragraph(f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}", styles['Normal']))
        story.append(Spacer(1, 0.3*inch))

        for i, gen in enumerate(generation_history, 1):
            story.append(Paragraph(f"<b>Documentation #{i}</b>", styles['Heading2']))
            story.append(Paragraph(f"<b>Style:</b> {gen['style'].upper()}", styles['Normal']))
            story.append(Spacer(1, 0.1*inch))

            story.append(Paragraph("<b>Original Code:</b>", styles['Normal']))
            story.append(Preformatted(gen['code'], code_style))
            story.append(Spacer(1, 0.15*inch))

            story.append(Paragraph("<b>Generated Docstring:</b>", styles['Normal']))
            story.append(Preformatted(gen['docstring'], code_style))
            story.append(Spacer(1, 0.3*inch))

            if i % 2 == 0 and i != len(generation_history):
                story.append(PageBreak())

        story.append(Spacer(1, 0.3*inch))
        story.append(Paragraph(f"Total Documentations: {len(generation_history)}", styles['Normal']))

        doc.build(story)
        return f"✅ PDF Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None
```

```python
def download_txt():
    """Download all generated docstrings as TXT"""
    if not generation_history:
        return "❌ No documentation generated!", None

    content = f"Doc-Genie Documentation Report\nGenerated: {datetime.now()}\n{'='*80}\n\n"

    for i, gen in enumerate(generation_history, 1):
        content += f"Documentation #{i}\n"
        content += f"Style: {gen['style'].upper()}\n"
        content += f"Generated: {gen['timestamp']}\n"
        content += f"{'-'*80}\n\n"
        content += f"ORIGINAL CODE:\n{gen['code']}\n\n"
        content += f"GENERATED DOCSTRING:\n{gen['docstring']}\n\n"
        content += f"{'-'*80}\n\n"

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.txt"

    try:
        Path(filename).write_text(content)
        return f"✅ TXT Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None

def generate_whatsapp_link():
    """Generate WhatsApp share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = "📚 *Doc-Genie Documentation*\n\n"
    for i, gen in enumerate(generation_history[-3:], 1):
        message += f"*Doc {i}:*\n"
        message += f"Style: {gen['style']}\n"
        message += f"`{gen['code'][:60]}...`\n\n"

    encoded = urllib.parse.quote(message)
    link = f"https://wa.me/?text={encoded}"

    return f"✅ WhatsApp Link:\n\n{link}\n\n📱 Share with team!"

def generate_facebook_link():
    """Generate Facebook share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = f"I generated {len(generation_history)} professional Python function docstrings with Doc-Genie! Check it out!"
    encoded = urllib.parse.quote(message)
    link = f"https://www.facebook.com/sharer/sharer.php?quote={encoded}"

    return f"✅ Facebook Link:\n\n{link}\n\n👍 Share on Facebook!"
```

```python
print("🚀 Loading Doc-Genie Improved Version...")

with gr.Blocks(
    title="Doc-Genie - Python Documentation Generator",
    theme=gr.themes.Soft(),
    css="""
    body { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .gradio-container { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .button-primary { background: linear-gradient(135deg, #FF6B6B 0%, #FF8E72 100%); }
    .button-secondary { background: linear-gradient(135deg, #4ECDC4 0%, #44A08D 100%); }
    .code-block { background: #0f3460; border: 2px solid #4ECDC4; border-radius: 8px; padding: 15px; }
    """
) as demo:

    gr.Markdown("""
    # 📚 Doc-Genie v2.0
    ## Professional Python Function Docstring Generator

    Transform your Python functions with AI-powered, accurate documentation!
    """)

    with gr.Row():
        with gr.Column(scale=3):
            # Code Input Area
            gr.Markdown("### 📝 Input Function Code")

            code_input = gr.Code(
                language="python",
                label="Python Function",
                lines=10,
                value="def calculate_factorial(n):\n    if n == 0:\n        return 1\n    return n * calculate_factorial(n-1)"
            )
```

Gradio Interface & User Experience

Interface Components Overview

Main Interface Sections:

1. LEFT COLUMN (Code Input/Output):
   - Code input area with syntax highlighting
   - File upload for .py files
   - Style selector (Google/NumPy radio)
   - Generate button
   - Code output display

2. RIGHT COLUMN (Controls & Export):
   - Clear history button
   - PDF download button
   - TXT download button
   - WhatsApp share button

- Facebook share button
- Feature information panel

Core Input/Output Interface

Code Input Component:
- Textarea with Python syntax highlighting
- Pre-filled with example function
- Support for multi-line code
- Line numbers and formatting

File Upload:
- Accept .py file format
- Read and display file content
- Error handling for invalid files

Style Selector:
- Radio buttons: "google" and "numpy"
- Real-time style switching
- Default: "google" style

Output Display:
- Syntax-highlighted code block
- Shows function with generated docstring
- Copy-paste ready format

```python
# File Upload
file_upload = gr.File(label="📁 Or Upload .py File", file_types=['.py'])
upload_status = gr.Textbox(label="Upload Status", interactive=False, lines=1)

def process_and_display(file):
    if file:
        content, status = process_upload(file)
        return content, status
    return "", "No file"

file_upload.change(process_and_display, inputs=[file_upload], outputs=[code_input, upload_status])

# Style Selection
with gr.Row():
    docstring_style = gr.Radio(["google", "numpy"], value="google", label="🎨 Docstring Style")
    generate_btn = gr.Button("✨ Generate Docstring", size="lg", variant="primary", scale=2)

# Output Area
gr.Markdown("### 📄 Generated Output")

code_output = gr.Code(
    language="python",
    label="Function with Docstring",
    lines=15,
    interactive=False
)
```

```python
            lines=15,
            interactive=False
        )

        gen_status = gr.Textbox(label="Status", interactive=False, lines=1)

        generate_btn.click(
            generate_docstring,
            inputs=[code_input, docstring_style],
            outputs=[code_output, gen_status]
        )

    with gr.Column(scale=1):
        gr.Markdown("### 🎮 Controls")

        clear_btn = gr.Button("🗑️ Clear History", variant="stop", size="sm")
        clear_btn.click(clear_history, outputs=[upload_status])

        gr.Markdown("---")
        gr.Markdown("### 📥 Export")

        pdf_btn = gr.Button("📄 Download PDF", size="sm", variant="primary")
        txt_btn = gr.Button("📋 Download TXT", size="sm", variant="primary")

        pdf_file = gr.File(label="PDF File", visible=True)
        txt_file = gr.File(label="TXT File", visible=True)
```

```python
        txt_btn.click(download_txt, outputs=[export_status, txt_file])

        gr.Markdown("---")
        gr.Markdown("### 📤 Share")

        whatsapp_btn = gr.Button("💬 WhatsApp", size="sm", variant="secondary")
        facebook_btn = gr.Button("👍 Facebook", size="sm", variant="secondary")

        share_output = gr.Textbox(label="Share Link", interactive=False, lines=4)

        whatsapp_btn.click(generate_whatsapp_link, outputs=[share_output])
        facebook_btn.click(generate_facebook_link, outputs=[share_output])

        gr.Markdown("---")
        gr.Markdown("""
        ### 📊 Features

        **Improvements:**
        - ✅ Accurate logic analysis
        - ✅ Contextual descriptions
        - ✅ Operation detection
        - ✅ Loop & condition recognition
        - ✅ Better PDF export
        - ✅ Consistent output
```

```python
        ### 🎯 Supported Styles

        **Google:** Standard format
        **NumPy:** Scientific format
        """)

print("\n✅ Doc-Genie v2.0 Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

Export & Sharing Features

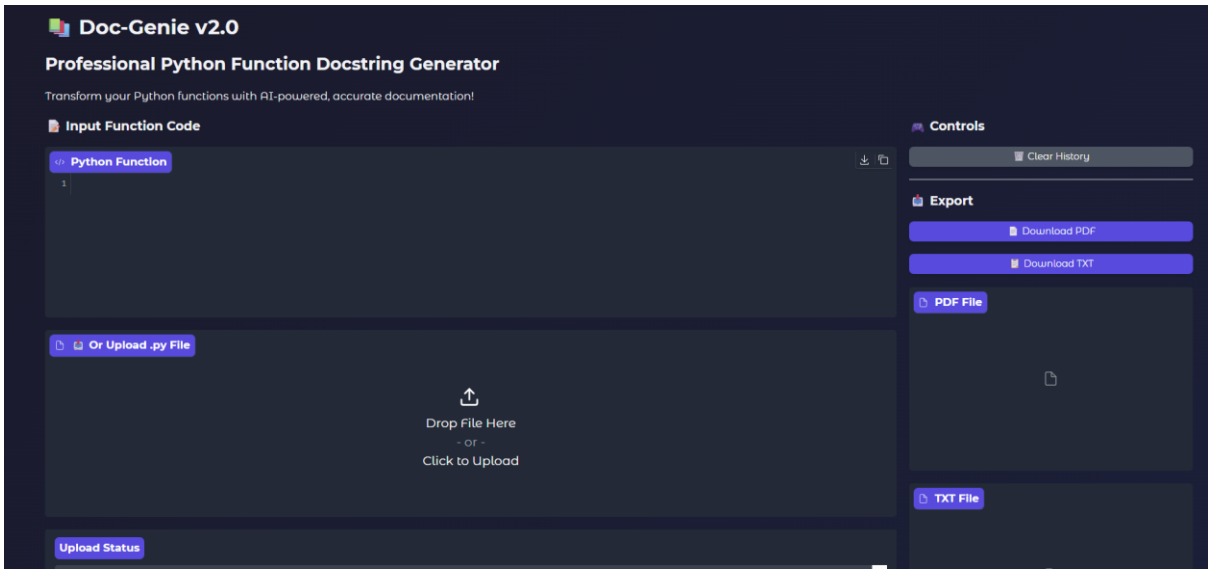
PDF Export:
- ReportLab-based document generation
- Professional styling and layout
- Timestamp and metadata inclusion
- Code block formatting
- Pagination for multiple docs

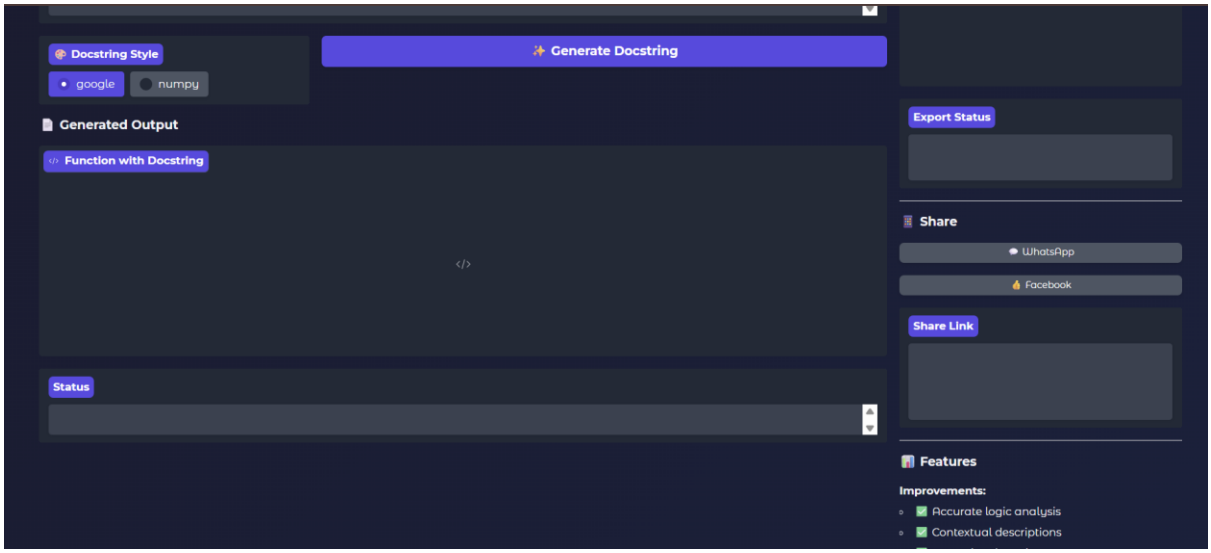
TXT Export:
- Plain text format with clear separators
- Includes original code and docstring
- Preserves formatting for readability
- Batch export support

Social Sharing:
- WhatsApp link with message preview
- Facebook share with summary
- URL encoding for special characters
- Message customization

Testing & Deployment

Google Colab Deployment





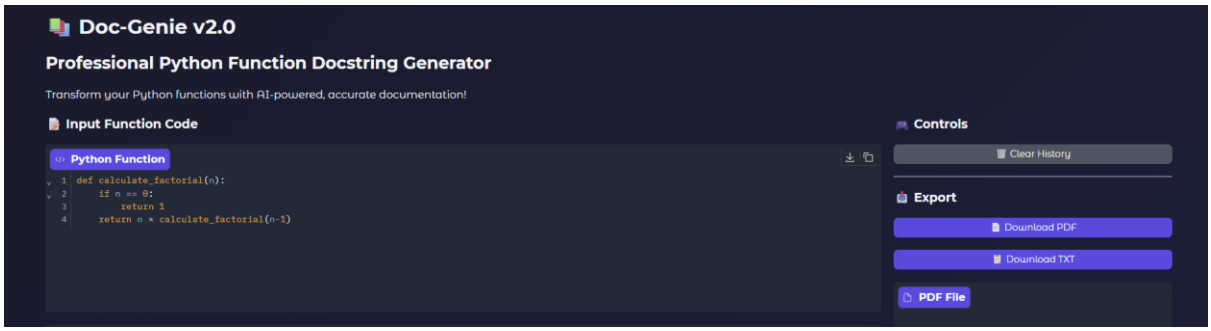
Public URL https://7dc8a63b8818beaab9.gradio.live

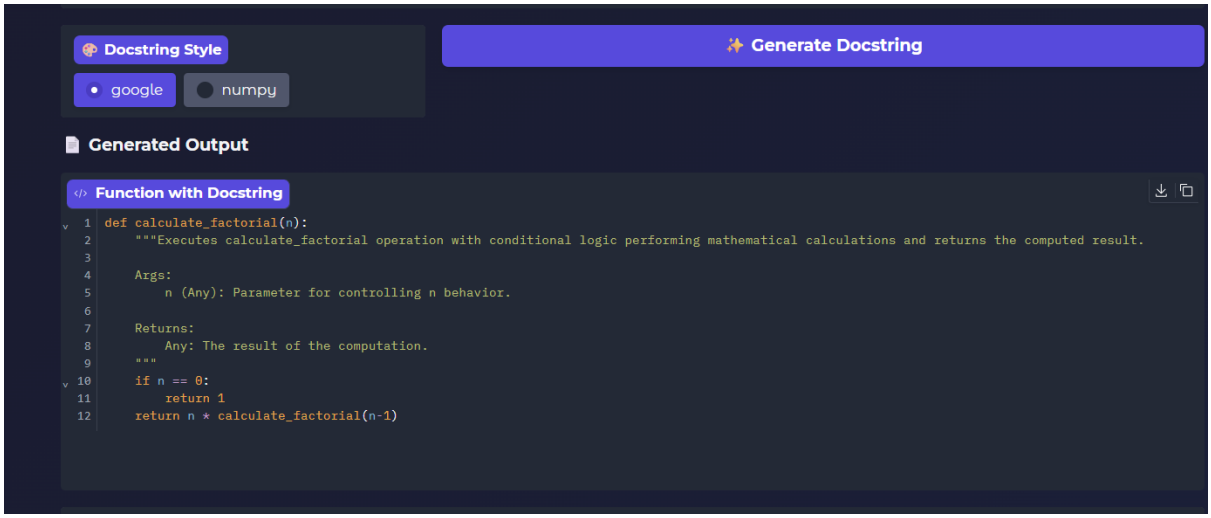
Functional Testing

Simple Function Documentation

Input Code:

```
def calculate_factorial(n):
    if n == 0:
        return 1
    return n * calculate_factorial(n-1)
```





Future Enhancements:

The modular architecture allows for extension with features such as support for additional programming languages (Java, C++, JavaScript), integration with IDE plugins (VSCode, PyCharm), advanced machine learning-based function description generation, support for class and module documentation, automatic documentation deployment to GitHub/GitLab, integration with documentation generators (Sphinx, MkDocs), and real-time code quality analysis with documentation metrics.

With continued development, Doc-Genie can evolve into a comprehensive code documentation platform supporting diverse programming paradigms while maintaining its focus on accuracy, ease of use, and professional documentation standards. The intelligent analysis engine provides a foundation for future features including AI-powered best practice recommendations, documentation style enforcement, and automated technical debt identification.

Conclusion

Doc-Genie - successfully delivers an intelligent Python documentation assistant that combines advanced AST analysis with intuitive documentation generation through Gradio's interactive interface. By leveraging Python's introspection capabilities and sophisticated code analysis algorithms, the system provides fast, accurate, and professional documentation generation accessible to

developers, teams, and educators.

The platform demonstrates the viability of intelligent code documentation, making professional Python documentation generation accessible while maintaining analysis accuracy, documentation quality, and support for industry-standard docstring formats. With advanced logic analysis, multiple documentation styles, comprehensive export capabilities, and social media sharing, Doc-Genie - empowers developers to maintain well-documented codebases efficiently.